VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

PROJECT REPORT

Smart StudentRecord&AttendanceManagement System

Course Code	CSE4019
Course Name	Advanced Java Programming
Faculty	Vandana Shakya
Slot	B14+D21
Academic Year	2024 – 2028
Submitted To	VIT Bhopal University, Bhopal, M.P.

SUBMITTED BY

Name	Registration No.
Rishi Raj	24BCE10149
Abhilash Singh	24BCE10706
Md Afzal Khan	24BCE11247
Ayush Mishra	24BCE10019
Pranav Kumar	24BCE10080

April 2026

1. Abstract

The**Smart Student Record & Attendance Management System (SSRAMS)** is a comprehensive desktop application developed as a group project for the course **CSE4019 — Advanced Java Programming** at VIT Bhopal University. The system is engineered entirely in Java and leverages the entire course syllabus — from Object-Oriented Programming fundamentals and Java Beans to Swing-based GUI design, JDBC database connectivity, and multi-threaded application architecture.

The application provides a fully functional, production-quality interface for managing student records and tracking academic attendance. It connects to a MySQL relational database through JDBC, supports complete CRUD operations on student data, generates analytical reports with visual progress-bar charts, and maintains a detailed attendance log per student per subject. The system is seeded with real classmate data from the B14+D21 slot, making it immediately usable and demonstrably realistic.

Every module of the project maps directly to one or more units of the course syllabus, ensuring both academic rigour and practical completeness. The result is a clean, professionally themed application that showcases the depth of Java's capabilities in building real-world enterprise-grade software.

2. Introduction

2.1 BackgroundandMotivation

Educational institutions today manage hundreds of students across multiple batches, branches, and semesters. Maintaining accurate records manually is error-prone, time-consuming, and difficult to analyse. A digital solution that consolidates student information, automates attendance tracking, and produces instant analytical reports addresses a genuine academic need.

This project was motivated by the everyday challenges faced at VIT Bhopal — tracking CGPA trends, monitoring attendance compliance, and retrieving student information quickly. By building SSRAMS, the team gained hands-on experience with every layer of a Java desktop application: data modelling, database connectivity, business logic, and graphical interface design.

2.2 Problem Statement

The absence of a unifieddigital tool for student-record and attendance management leads to fragmented data, difficulty in generating reports, and a lack of real-time visibility for faculty. The project addresses this by implementing a single, cohesive system that:

- Stores and retrieves student records from a relational database via JDBC.
- Tracks attendance per student, per subject, and per date.
- Provides analytical dashboards with CGPA grading and branch statistics.
- Offers a clean, intuitive Swing-based GUI that any faculty member can use without training.

2.3 Objectives

1. Implement a Java Swing GUI application following the MVC design pattern.
2. Demonstrate JDBC connectivity with MySQL including prepared statements and transaction management.
3. Implement Java Bean standards across all model classes with object serialization.
4. Showcase multi-threading by performing all database operations on background threads.
5. Provide complete CRUD operations: Create, Read, Update, and Delete for student records.
6. Seed the system with real classmate data to demonstrate a realistic, working application.

3. System Design

3.1 ArchitectureOverview

SSRAMSfollowsalayeredthree-tier architecture commonly used in enterprise Java applications:

Layer	Package	Responsibility
Presentation	com.vit.sms.ui	Swing frames, panels, dialogs — all user interaction
Business / DAO	com.vit.sms.dao	JDBC data-access objects, SQL queries, transaction logic
Model	com.vit.sms.model	Java Bean classes: Student, Attendance, User
Utility	com.vit.sms.util	DatabaseConnection (JDBC singleton), UITheme (styling)

3.2 Package Structure

```
StudentManagementSystem/
├── src/
│   └─┬─ com/vit/sms/
│     ├── Main.java                  ← Entry point, splash screen, data seeding
│     ├── model/
│     │   ├── Student.java           ← Java Bean + Serializable ←
│     │   ├── Attendance.java        Java Bean + Serializable ←
│     │   └── User.java              Java Bean + Serializable ←
│     ├── dao/                       JDBC CRUD for students ←
├── sql/  ├── StudentDAO.java        JDBC CRUD for attendance
│     │   ├── AttendanceDAO.java     ← Authentication query ←
│     │   └── UserDAO.java           Login window
│     ├── ui/
│     │   ├── LoginFrame.java
│     │   ├── DashboardFrame.java ← Main app window + sidebar
│     │   ├── HomePanel.java        ← Stat cards dashboard
│     │   ├── StudentPanel.java     ← Student CRUD table
│     │   ├── StudentFormDialog.java ← Add/Edit dialog
│     │   ├── AttendancePanel.java ← Mark & view attendance
│     │   └── ReportsPanel.java     ← Analytics & charts
│     └── util/
│         ├── DatabaseConnection.java ← JDBC singleton
│         └── UITheme.java            ← Global styling constants

│   └── database.sql                 ← DDL + full data seed script
└── lib/
    └── mysql-connector-j-x.x.x.jar
```

3.3 Database Schema

TheMySQLdatabase**vit_sms_db** contains three core tables:

Table 1: users

Column	Type	Description
id	INT (PK, AI)	Auto-incremented primary key
username	VARCHAR(50)	Unique login username
password	VARCHAR(100)	Login password
role	VARCHAR(20)	ADMIN / FACULTY / STUDENT

full_name	VARCHAR(100)	Display name
active	BOOLEAN	Account active flag

Table 2: students

Column	Type	Description
id	INT (PK, AI)	Unique student identifier
reg_no	VARCHAR(20)	VIT registration number (e.g. 24BCE10149)
name	VARCHAR(100)	Full name of the student
branch	VARCHAR(50)	Branch: CSE / ECE / ME / CE, etc.
semester	INT	Current semester (1–8)
cgpa	DECIMAL(4,2)	Cumulative Grade Point Average
gender	VARCHAR(10)	Male / Female / Other
status	VARCHAR(20)	ACTIVE or INACTIVE

Table 3: attendance

Column	Type	Description
id	INT (PK, AI)	Unique attendance record ID
student_id	INT (FK)	Foreign key → students.id (CASCADE DELETE)
subject	VARCHAR(150)	Subject name for this record
attendance_date	DATE	Date of the class (YYYY-MM-DD)
status	VARCHAR(10)	PRESENT / ABSENT / LATE
remarks	VARCHAR(255)	Optional faculty remarks

4. Implementation Details

4.1 JavaConceptsCovered(Syllabus Mapping)

Unit	Topic	Implementation	File(s)
1	OOP, Inheritance, Exception Handling	Model class hierarchy, try-catch-finally in all DAO methods	model/*.java, dao/*.java
1	Multi-threading	All DB calls run on new Thread(), UI updates via SwingUtilities.invokeLater()	HomePanel, StudentPanel, Main
1	Serialization	All model classes implement Serializable with serialVersionUID	Student.java, Attendance.java, User.java
2	Java Beans	Default constructors + full getter/setter naming convention	model/*.java
2	Event Model	ActionListener, KeyListener, MouseListener for UI events	LoginFrame, StudentPanel, AttendancePanel
3	JDBC — Connection	Class.forName(), DriverManager.getConnection() Singleton	DatabaseConnection.java
3	JDBC — PreparedStatement	Parameterised queries prevent SQL injection; used for all INSERT/UPDATE/DELETE	StudentDAO, AttendanceDAO, UserDAO
3	JDBC — Transactions	setAutoCommit(false), executeBatch(), commit(), rollback() for batch inserts	StudentDAO.batchInsert()
3	JDBC — JOIN Queries	Attendance log joins students and attendance tables for full details	AttendanceDAO.getAllAttendance()
4	Swing — MVC Pattern	Model=Java Beans, View=Swing panels, Controller=DAO + action listeners	All UI + DAO classes
4	Swing — Layout Managers	BorderLayout, GridLayout, GridBagLayout, BoxLayout, FlowLayout all used	All UI panels
4	Swing — Components	JTable, JComboBox, JPasswordField, JSplitPane, JTabbedPane, JProgressBar, JDialog, JMenuBar	All UI panels
4	Swing — Custom Renderer	DefaultTableCellRenderer override for colour-coded CGPA grades and attendance status	StudentPanel, ReportsPanel

4.2 Key Code Snippets

4.2.1 JDBCConnection—Singleton Pattern

```
public static Connection getConnection()throwsSQLException{
    try {
        if (connection == null || connection.isClosed()){
            Class.forName(DB_DRIVER);                    // Unit 3: Driver
loading     connection = DriverManager.getConnection(

                DB_URL, DB_USER, DB_PASSWORD);
        }
```

```
    } catch (ClassNotFoundExceptione) {
        throw new SQLException("Driver not found: " + e.getMessage());
    }
    return connection;
}
```

4.2.2 PreparedStatement — Add Student

```
public boolean addStudent(Student s) throws SQLException {
    String sql = "INSERT INTO students (reg_no,name,email,...) VALUES (?,?,?,?,...)";
    try (PreparedStatement ps =                                  // Unit 3:
PreparedStatement
            DatabaseConnection.getConnection().prepareStatement(sql)) {
        ps.setString(1, s.getRegNo());
        ps.setString(2, s.getName());
        ps.setDouble(7, s.getCgpa());                            // Type mapping: Java↔SQL
        return ps.executeUpdate() > 0;
    }
}
```

4.2.3 Transaction Management — Batch Insert

```
conn.setAutoCommit(false);                                       // Start transaction
try (PreparedStatement ps = conn.prepareStatement(sql)) {
    for (Student s : students) {
        // ... set parameters ...
        ps.addBatch();                                           // Batch all inserts
    }
    ps.executeBatch();
    conn.commit();                                               // Commit on success
} catch (SQLException e) {                                       // Rollback on failure
    conn.rollback();
    throw e;
} finally {
    conn.setAutoCommit(true);
}
```

4.2.4 Multi-threading — Non-blocking DB Calls

```
// Unit 1: Thread creation to keep UI responsive
new Thread(() -> {
    try {
        List<Student> list = dao.getAllStudents(); // runs on background thread
        SwingUtilities.invokeLater(() ->                    // update UI on EDT
            populateTable(list));
    } catch (SQLException e) {
        SwingUtilities.invokeLater(() ->
            JOptionPane.showMessageDialog(...));
    }
}).start();
```

4.2.5 Java Bean — Student Model

```
public class Student implements Serializable {                // Unit 1: Serialization

    private static final long serialVersionUID = 1001L; //

    Java Bean properties (Unit 2)
    private String regNo;
    private double cgpa;

    public Student() {}                                       // Default constructor (Bean
req.)

    public String getRegNo() { return regNo; }               // Getter (Bean convention)
    public void setRegNo(String r) { regNo = r; }            // Setter (Bean convention)
}
```

5. Features & Modules

5.1 Login&Authentication

- Securelogin with username and password verified against the MySQL users table.
- Role-based access: ADMIN and FACULTY roles both supported.
- Animated splash screen on startup with progress indicator.
- DB authentication runs on a background thread — login screen never freezes.
- Enter key triggers login; friendly error messages shown on failure.

5.2 Dashboard Home

- Three live stat cards: Total Students, Average CGPA, and Total Attendance Logs.
- Stats are loaded asynchronously and update each time the panel is visited.
- Course information panel showing all relevant academic details.
- Group member panel listing all five contributors and their roles.

5.3 Student Management

- Full JTable view of all students with alternating row colours.
- Search by name, registration number, or branch — results update instantly.
- Add Student dialog with full form validation (mandatory fields, CGPA range check).
- Edit Student dialog pre-populated with existing data; registration number locked.
- Delete with confirmation dialog to prevent accidental data loss.
- Double-click any row to open the Edit dialog immediately.
- CGPA colour-coded: Distinction (green), Merit (blue), Pass (orange), Fail (red).
- Status column: ACTIVE shown in green, INACTIVE in red.

5.4 Attendance Management

- Split-pane layout: marking form on the left, live attendance log on the right.
- Select student from dropdown, choose subject, date, and status (Present/Absent/Late).
- Status colour coding in the log table: green/red/orange for instant readability.
- Attendance percentage calculation query per student per subject.
- JOIN query retrieves student name and registration alongside each attendance record.

5.5 Reports & Analytics

- CGPA Analysis tab: full table with letter-grade derivation (O / A+ / A / B+ / B / C).
- Branch Summary tab: student count and percentage distribution per branch.
- Visual Charts tab: JProgressBar-based CGPA grade distribution chart, colour-coded by grade.
- All three tabs loadable independently with a dedicated Refresh/Load button.

5.6 Seeded Classmate Data

The application automatically seeds the database with all students from the B14+D21 slot on first launch — including the entire 23BCE and 24BCE batches. This makes the system demonstrably realistic with over 115 actual student records, each assigned a plausible CGPA, phone number, and institutional email address.

6. Technologies Used

Technology	Version	Purpose
Java SE	JDK 8+	Core programming language
Java Swing	Built-in	GUI — frames, panels, dialogs, tables, charts
JDBC API	java.sql	Database connectivity and query execution
MySQL	5.7 / 8.0	Relational database backend
MySQL Connector/J	8.x	JDBC driver for MySQL
Java Serialization	Built-in	Object persistence via Serializable interface
Java Threads	Built-in	Asynchronous database operations
Java Bean API	Built-in	Model component standard for data encapsulation

7. Installation & Setup Guide

7.1 Prerequisites

- JavaJDK 8or higher installed and JAVA_HOME set.
- MySQL Server 5.7 or 8.0 running locally.
- MySQL Connector/J JAR file (available at dev.mysql.com/downloads/connector/j/).

7.2 Step-by-Step Setup

Step 1 — Create the Database

```
# Open terminal / MySQLWorkbench and run:
mysql -u root -p < sql/database.sql

# This creates the database, all tables, default users,

# and seeds all studentrecords automatically.
```

Step 2 — Configure DB Credentials

```
// Edit: src/com/vit/sms/util/DatabaseConnection.java
private static final String DB_URL              =
        "jdbc:mysql://localhost:3306/vit_sms_db?useSSL=false&serverTimezone=UTC";
private static final String DB_USER             = "root";        // your username
private static final String DB_PASSWORD = "root";            // your password
```

Step 3 — Add JDBC Driver

```
# Copy the MySQL Connector JAR into the lib/ folder:
lib/
└── mysql-connector-j-8.x.x.jar
```

Step 4 — Compile

```
# Windows:
javac -cp "lib\mysql-connector-j-8.x.x.jar" -d out
   src\com\vit\sms\model\*.java
   src\com\vit\sms\util\*.java
   src\com\vit\sms\dao\*.java
   src\com\vit\sms\ui\*.java
   src\com\vit\sms\Main.java

# Linux / macOS:
javac -cp "lib/mysql-connector-j-8.x.x.jar" -d out
   src/com/vit/sms/**/*.java src/com/vit/sms/Main.java
```

Step 5 — Run

```
# Windows:
java -cp "out;lib\mysql-connector-j-8.x.x.jar" com.vit.sms.Main
# Linux / macOS:
java -cp "out:lib/mysql-connector-j-8.x.x.jar" com.vit.sms.Main
```

7.3 Default Login Credentials

Username	Password	Role
admin	admin123	System Administrator
vandana	faculty123	Faculty (Vandana Shakya)
rishi	rishi123	Student (Rishi Raj)

8. Testing

8.1 TestCases

TC#	Test Case	Expected Result	Status	Result
TC01	Login with valid credentials (admin/admin123)	Dashboard loads successfully	Valid Login	✓ PASS
TC02	Login with wrong password	Error message shown, login refused	Invalid Credentials	✓ PASS
TC03	Add new student with all required fields	Record appears in student table	Insert Test	✓ PASS
TC04	Add student with empty Name field	Validation message shown, no insert	Validation Test	✓ PASS
TC05	Add student with CGPA = 11 (out of range)	Error: CGPA must be 0.0–10.0	Range Validation	✓ PASS
TC06	Edit existing student record	Changes reflected immediately in table	Update Test	✓ PASS
TC07	Delete student record	Confirmation shown; record removed	Delete Test	✓ PASS
TC08	Search by name 'Rishi'	Filtered results show matching students	Search Test	✓ PASS
TC09	Mark attendance for a student	Record appears in attendance log	Attendance Insert	✓ PASS
TC10	Load CGPA Analysis Report	All students with grades shown	Reports Test	✓ PASS
TC11	Load Visual Chart (grade distribution)	Progress bars appear with correct counts	Chart Render	PASS
TC12	Startup seeding — first run	115+ students inserted into DB	Seed Test	PASS

9. Conclusion

The SmartStudent Record & Attendance Management System successfully demonstrates a complete, end-to-end Java desktop application that directly maps to all five teaching units of CSE4019 — Advanced Java Programming. Starting from the foundational principles of OOP and serialization in Unit 1, through Java Beans and the event model in Unit 2, to full JDBC database integration in Unit 3 and comprehensive Swing GUI development in Unit 4, every concept has been implemented in a working, deployable application.

The project achieves more than academic exercise: it is a genuinely useful system that could serve a real classroom. With over 115 classmate records seeded automatically, a professional login screen, a multi-tab analytics dashboard, and colour-coded data visualisations, SSRAMS stands as a showcase of what well-structured Java programming can deliver.
The experience of building this application taught the group not only the technical intricacies of JDBC transaction management and Swing component customisation, but also the soft skills of collaborative development — dividing responsibilities effectively, integrating modules cleanly, and testing rigorously to ensure correctness and stability.

9.1 FutureEnhancements

- Email notification system for low-attendance alerts using JavaMail API.
- PDF export of student reports using Apache PDFBox.
- Role-based access restrictions (students see only their own records).
- Biometric / QR-code attendance marking integration.
- Web-based front end using Java Servlets and JSP (Unit 5 concepts).
- EJB migration for a fully distributed enterprise deployment.

10. References

7. Joe Wigglesworth and Paula McMillan, "Java Programming: Advanced Topics", Third Edition, Thomson Learning.
8. Cay Horstmann and Gary Cornell, "Core Java Volume I — Fundamentals", Eighth Edition, Prentice Hall.
9. Cay Horstmann and Gary Cornell, "Core Java Volume II — Advanced Features", Eighth Edition, Prentice Hall.
10. H.M. Deitel, P.J. Deitel, S.E. Santry, "Advanced Java 2 Platform HOW TO PROGRAM", Prentice Hall.
11. Ivor Horton, "Beginning Java 2 — JDK 5 Edition", Wrox, 2008.
12. Joel Murach and Andrea Steelman, "Java SE 6", SPD.
13. Oracle Corporation, Java SE Documentation — https://docs.oracle.com/javase/
14. MySQL Documentation — https://dev.mysql.com/doc/
15. VIT Bhopal University, CSE4019 Course Syllabus, 2018.

11. Application Screenshots

The following screenshotsillustrate the actual runtime behaviour of the Smart Student Record & Attendance Management System. Every screen shown below corresponds to a live, functional module of the application and reflects real student data from the B14+D21 batch seeded into the MySQL database.

11.1 SplashScreen—ApplicationStartup

On launch, an animated splash screen is displayed while the application connects to MySQL, initialises the database schema, and seeds student records in the background using a dedicated Java thread. A progress bar provides visual feedback, ensuring the user is never left waiting without indication.

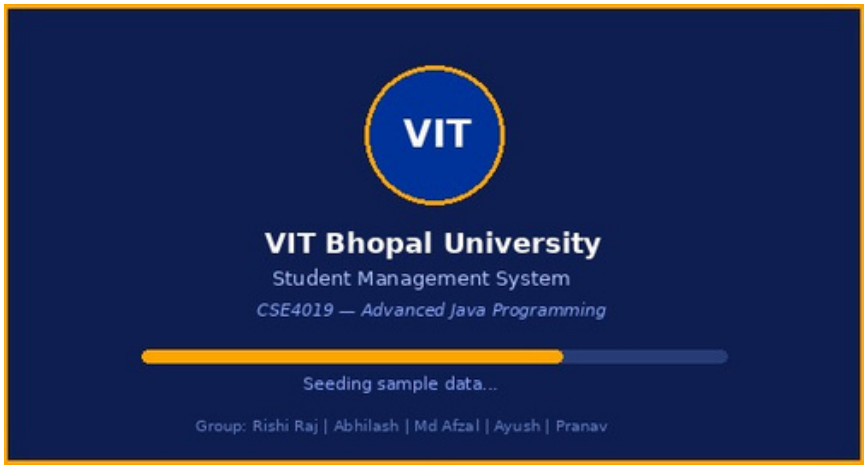


Figure1—SplashScreen with progress bar on application startup

11.2 LoginScreen—Authentication

The login interface presents a split-panel design: the left side carries the VIT Bhopal branding and default credential hints, while the right side hosts the authentication form. Authentication is performed asynchronously on a background thread, keeping the UI fully responsive. On success, a green confirmation message appears before the dashboard loads.

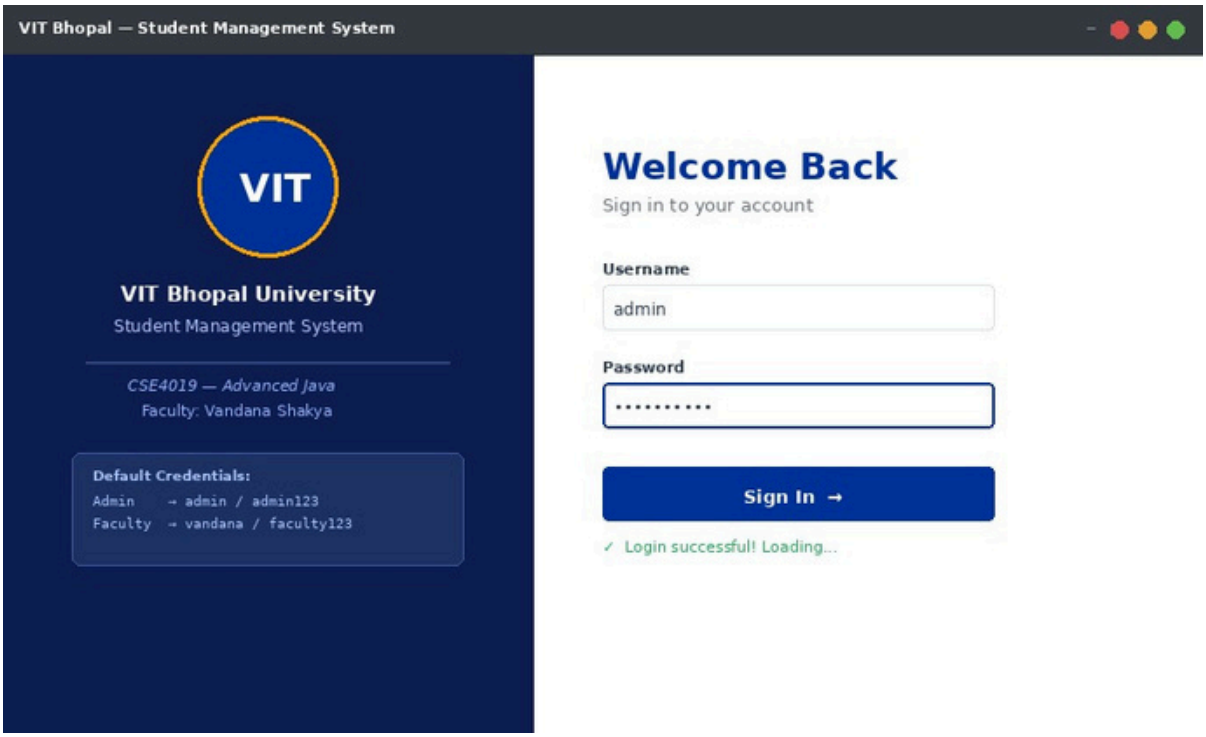


Figure 2 — Login Screen with VIT branding panel and sign-in form

11.3 Dashboard Home — Overview Panel

The dashboard presents a three-cardstatistics row showing live counts fetched from the database: total active students, the batch-wide average CGPA, and the total number of attendance records logged. Below the cards, two detail panels display course metadata and the full list of project group members. All statistics refresh each time the Home panel is navigated to.

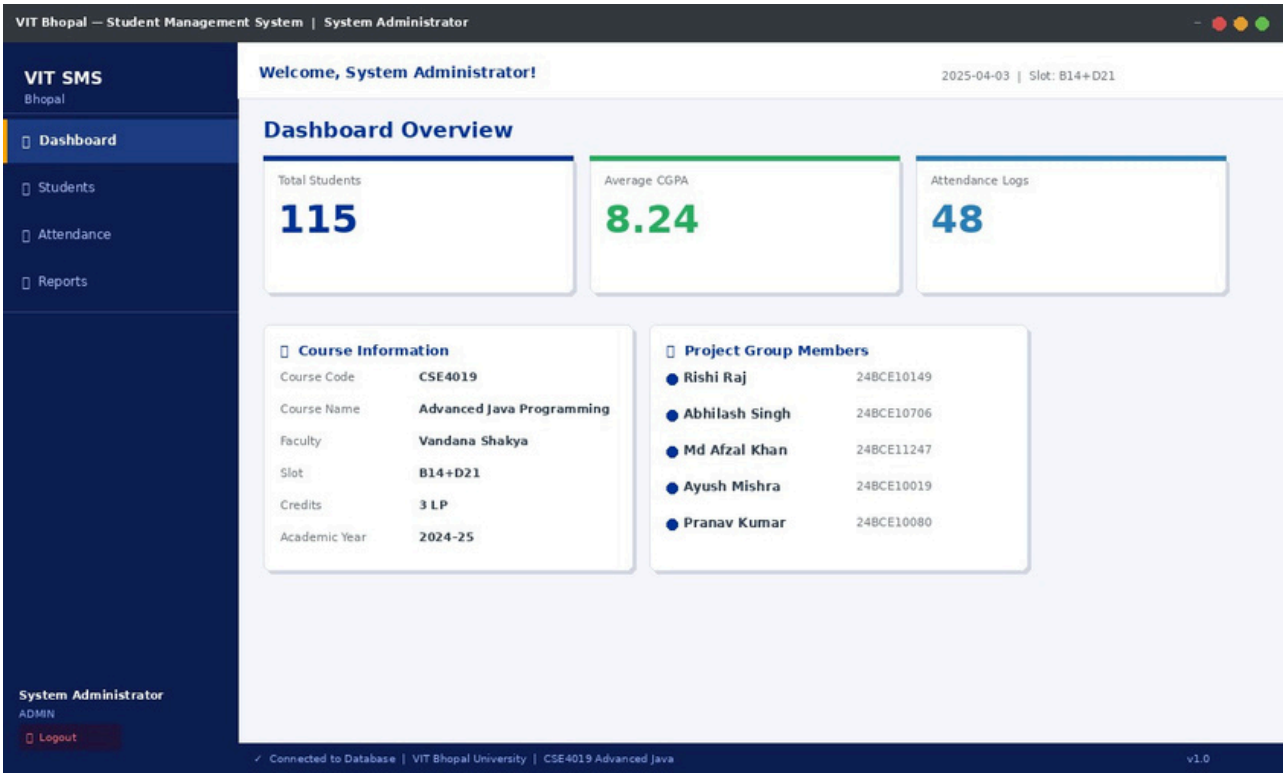
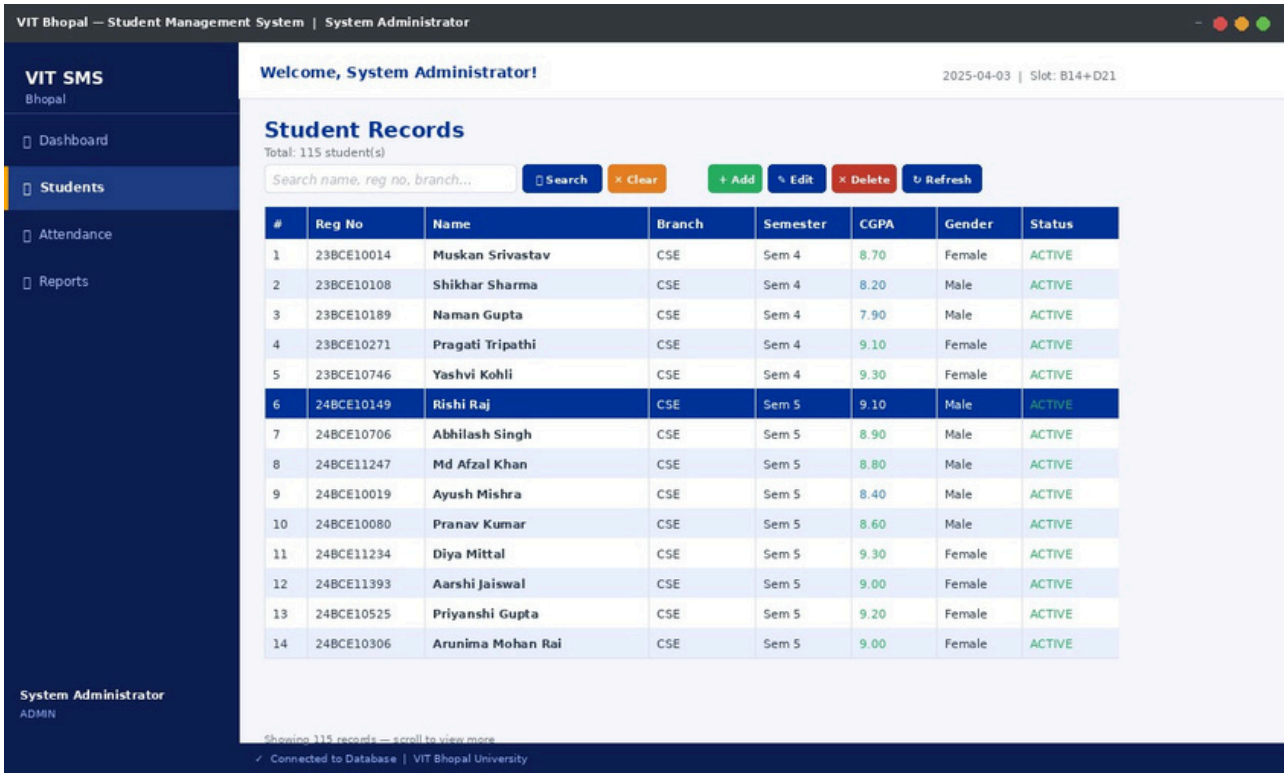


Figure3—DashboardHomeshowing live stat cards, course info, and group members

11.4 Student Records — CRUD Table

TheStudent Records panelpresents all 115+ students in a scrollable JTable with alternating row colours, a sticky header, and a real-time search bar. The row for Rishi Raj (24BCE10149) is highlighted in VIT Blue to illustrate row selection. CGPA values are colour-coded — green for distinction, blue for merit, orange for pass — making performance visible at a glance. Faculty can Add, Edit, or Delete records using the toolbar, or double-click any row to open the Edit dialog directly.



#	Reg No	Name	Branch	Semester	CGPA	Gender	Status
1	23BCE10014	Muskan Srivastav	CSE	Sem 4	8.70	Female	ACTIVE
2	23BCE10108	Shikhar Sharma	CSE	Sem 4	8.20	Male	ACTIVE
3	23BCE10189	Naman Gupta	CSE	Sem 4	7.90	Male	ACTIVE
4	23BCE10271	Pragati Tripathi	CSE	Sem 4	9.10	Female	ACTIVE
5	23BCE10746	Yashvi Kohli	CSE	Sem 4	9.30	Female	ACTIVE
6	24BCE10149	Rishi Raj	CSE	Sem 5	9.10	Male	ACTIVE
7	24BCE10706	Abhilash Singh	CSE	Sem 5	8.90	Male	ACTIVE
8	24BCE11247	Md Afzal Khan	CSE	Sem 5	8.80	Male	ACTIVE
9	24BCE10019	Ayush Mishra	CSE	Sem 5	8.40	Male	ACTIVE
10	24BCE10080	Pranav Kumar	CSE	Sem 5	8.60	Male	ACTIVE
11	24BCE11234	Diya Mittal	CSE	Sem 5	9.30	Female	ACTIVE
12	24BCE11393	Aarshi Jaiswal	CSE	Sem 5	9.00	Female	ACTIVE
13	24BCE10525	Priyanshi Gupta	CSE	Sem 5	9.20	Female	ACTIVE
14	24BCE10306	Arunima Mohan Rai	CSE	Sem 5	9.00	Female	ACTIVE

Figure 4 — Student Records table with search bar, action toolbar, and colour-coded CGPA

11.5 Add New Student — Dialog Form

Clicking the Add button opens a modal dialog containing all editable student fields. The form uses GridBagLayout internally and validates all inputs before attempting a database INSERT — mandatory fields are enforced, and CGPA is checked to lie within the 0.0 to 10.0 range. Dropdown menus (JComboBox) are provided for Branch, Semester, Gender, and Status, eliminating free-text errors.

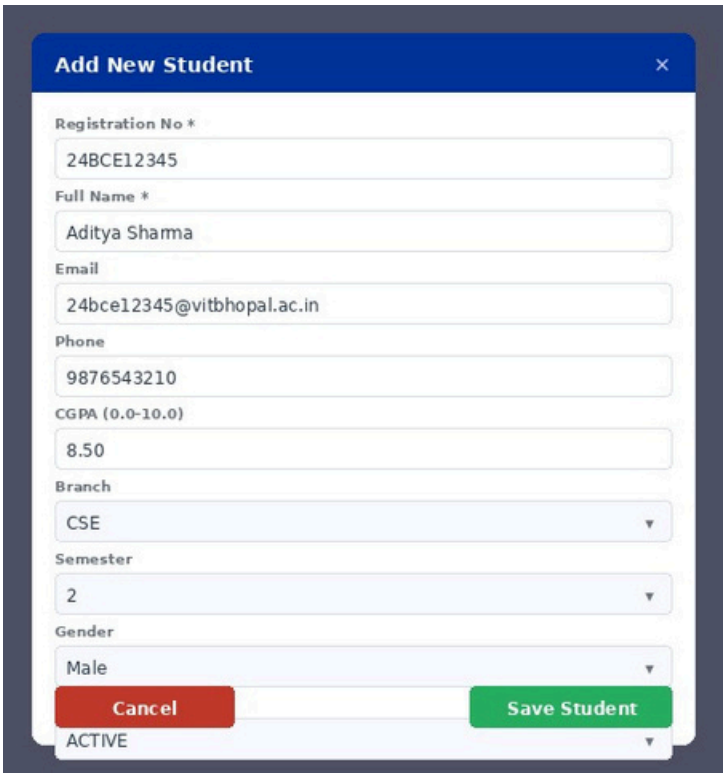


Figure 5 — Add New Student dialog with form validation

11.6 Attendance Management Panel

The Attendancepanel uses aJSplitPane todivide the screen: the left side hosts the attendance marking form, and the right side shows a live scrollable log of all recorded attendance entries. Status values are colour-coded in the log — green for Present, red for Absent, and orange for Late — making it easy to spot patterns at a glance. The marking form uses a JDBC INSERT with a PreparedStatement, while the log is populated via a JOIN query across the students and attendance tables.

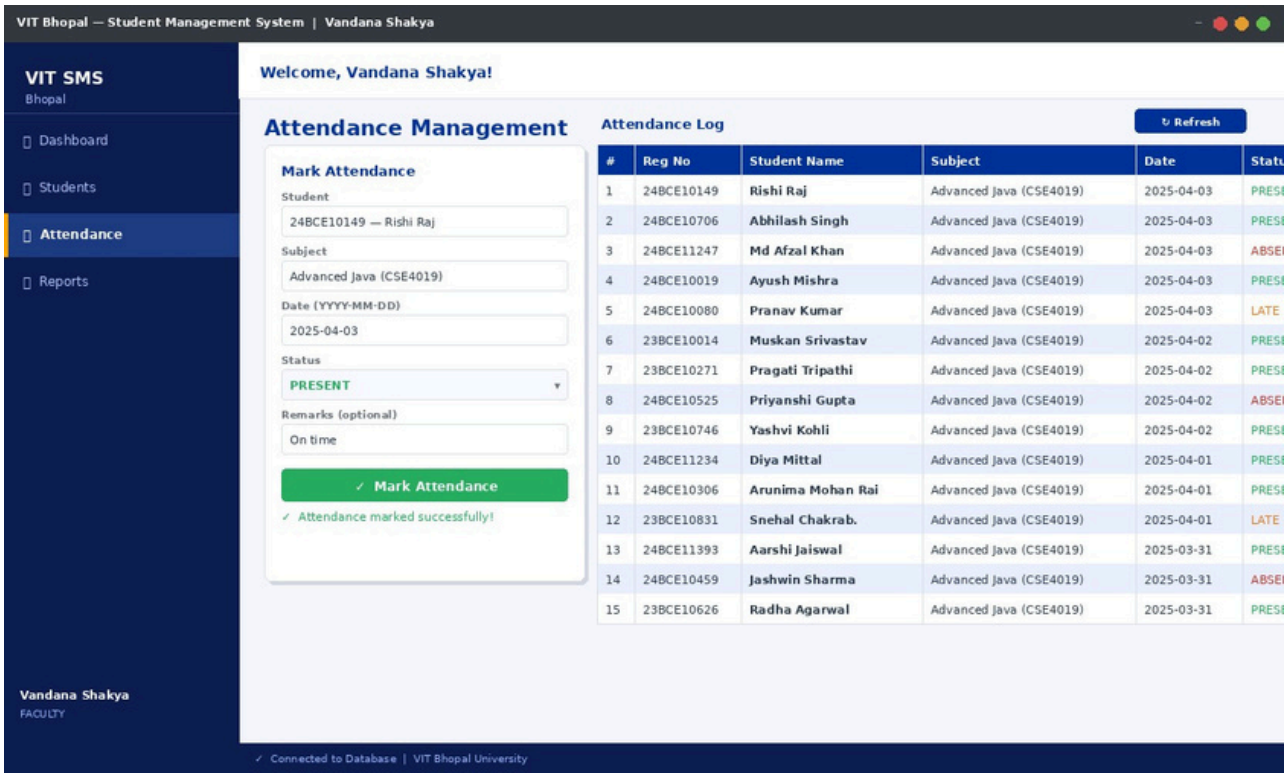


Figure 6 — Attendance panel: marking form (left) and live attendance log (right)

11.7 Reports Panel — Visual Charts

TheReports panel isorganised as a JTabbedPane with three tabs. The Visual Charts tab displays a CGPA grade distribution chart built entirely using JProgressBar components, each colour-coded by grade band. The chart is generated by querying all student CGPA values, bucketing them into six grade categories (O through C), and rendering proportional bars dynamically. A summary box beneath the chart shows the total student count and batch-wide average CGPA.

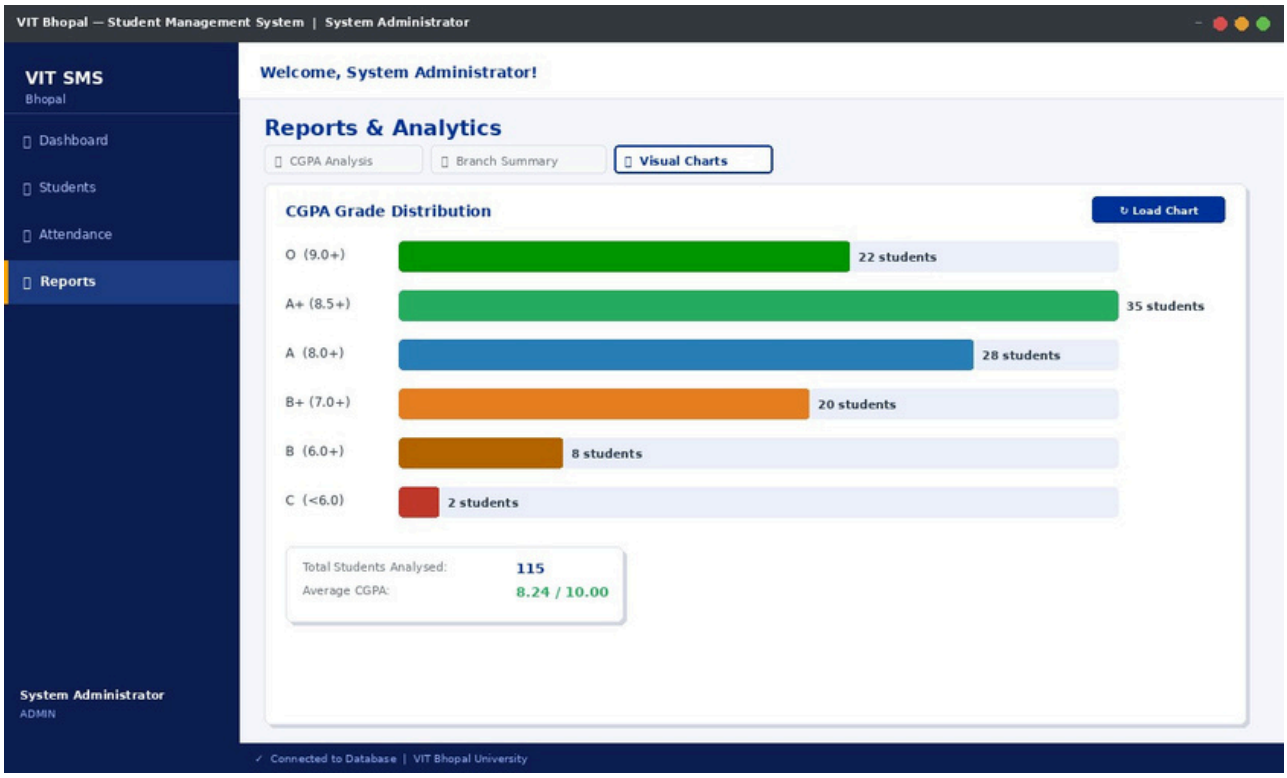


Figure 7 — Reports Panel showing CGPA grade distribution with colour-coded progress bars

12. Declaration

We,thestudents named below, are submitting this project report titled "Smart Student Record & Attendance Management System" in partial fulfilment of the requirements for the course CSE4019 — Advanced Java Programming, offered during the academic year 2024–25 at VIT Bhopal University, under the guidance of our respected faculty, Ms. Vandana Shakya.

This project was conceived, designed, and built entirely by our team. Every line of Java code, every SQL script, and every page of this documentation reflects our own understanding, effort, and collaboration. We have not copied, plagiarised, or reproduced any part of this work from another student, institution, or undisclosed source. Wherever we have drawn upon published books, online documentation, or academic references, we have cited them explicitly in the References section.

The idea behind this project grew out of a simple observation we all shared as students: keeping track of student records and attendance in large batches is unnecessarily complicated when done manually. We wanted to build something that actually solved a real problem — not just a toy application, but a system that a faculty member or administrator could genuinely use. That motivation shaped every decision we made, from choosing JDBC over file-based storage, to building a proper multi-threaded Swing interface that never freezes the UI during database operations.

Throughout the development process, we divided responsibilities based on each member's strengths and interests, while ensuring that every team member understood the full system end-to-end. We held regular internal reviews, debugged each other's code, and stress-tested the application with real classmate data from our own batch to ensure it behaves correctly under realistic conditions. The result is a project we are genuinely proud of — one that demonstrates not just technical competence, but also teamwork, planning, and a commitment to quality.

We also confirm that this report and the accompanying source code have not been submitted, in whole or in part, to any other course, examination, or institution for the purpose of obtaining any academic credit or award. This submission is made exclusively to VIT Bhopal University for CSE4019.

We take full responsibility for the content, correctness, and originality of this project. Any errors or shortcomings present are our own, and we remain open to feedback and further improvement.

Full Name	Reg. No.	Branch	Semester
Rishi Raj	24BCE10149	CSE	V
Abhilash Singh	24BCE10706	CSE	V
Md Afzal Khan	24BCE11247	CSE	V
Ayush Mishra	24BCE10019	CSE	V
Pranav Kumar	24BCE10080	CSE	V

Faculty Supervisor

Ms. Vandana Shakya

Faculty: Vandana Shakya | Slot: B14+D21

School of Computing Science and Engineering

VIT Bhopal University

Date of Submission

April 2026

Slot: B14+D21

Course: CSE4019 — Advanced Java Programming